mbusgw-t2r

Руководство пользователя

Номер документа: DO-T2RUG-RU-01A
Дата публикации: Август 2026
Сведения о ревизии: Это новое руководство.
Операционная система: Linux (glibc), ядро 4.x и новее
Версия программы: mbusgw-t2r X.00-10

NaPi World & StarLet Squad collaboration

mbusgw-t2r — Руководство пользователя

Copyright © 2026 NaPi World & StarLet Squad collaboration

Правовая информация

Сведения в настоящем документе могут быть изменены без предварительного уведомления. NaPi World & StarLet Squad collaboration не несёт ответственности за технические или редакционные ошибки и пропуски в настоящем документе.

Программа, описанная в настоящем руководстве, распространяется в надежде, что она будет полезной, но БЕЗ КАКИХ-ЛИБО ГАРАНТИЙ, в том числе без подразумеваемых гарантий товарной пригодности и пригодности для конкретной цели.

MODBUS — зарегистрированный товарный знак Schneider Electric USA, Inc. Linux — зарегистрированный товарный знак Линуса Торвальдса. UNIX — зарегистрированный товарный знак The Open Group.

Содержание

Предисловие iv

1. О настоящем руководстве iv

2. Для кого написано iv

3. Структура документа iv

4. Смежные документы iv

5. Соглашения iv

Глава 1. Что делает шлюз 1

Глава 2. Перед началом 2

Глава 3. Файл настроек 3

3.1. Раздел serials 3

3.2. Раздел listeners 3

Глава 4. Запуск шлюза 5

4.1. Первая проверка с mbpoll 5

Глава 5. Псевдоустройство меток времени 6

Глава 6. Как читать журнал 7

Глава 7. Поиск неисправностей 8

7.1. Симптом, причина, действие 8

7.2. Справочник кодов сообщений 9

7.3. Если ничего не помогло 9

Предисловие

1. О настоящем руководстве

Настоящее руководство описывает настройку, эксплуатацию и поиск неисправностей mbusgw-t2r — шлюза, который переводит запросы MODBUS TCP, приходящие из сети, в транзакции MODBUS RTU на последовательной линии (RS-485 или RS-232) и возвращает ответы обратно.

2. Для кого написано

Руководство рассчитано на читателя без опыта работы с MODBUS и администрирования Linux: каждый шаг расписан, ничего не подразумевается. Опытный инженер может сразу перейти к главе 3 (файл настроек) и главе 7 (поиск неисправностей).

3. Структура документа

Глава 1 объясняет, что делает шлюз. Глава 2 — контрольный список перед запуском. Глава 3 описывает файл настроек ключ за ключом. Глава 4 — запуск, опции командной строки и первая сквозная проверка. Глава 5 — встроенное псевдоустройство меток времени. Глава 6 учит читать журнал. Глава 7 — справочник по поиску неисправностей: симптом, причина, действие, и справочник всех кодов сообщений.

4. Смежные документы

README.md — последовательность установки (пакет StarLet, сборка, `make install`). MODBUS over Serial Line Specification and Implementation Guide и MODBUS Application Protocol Specification — сам протокол, доступны на modbus.org.

5. Соглашения

Обозначение	Значение
`Моноширинный`	Примеры кода, имена файлов, команды и экранный вывод.
`%T2R-E-КОД`	Код сообщения, как он выглядит в журнале шлюза; см. раздел 7.2.
`<...>`	Место подстановки фактического значения, например имени устройства.
`$`	Приглашение оболочки непривилегированного пользователя; вводить его не нужно.
`Ctrl/C`	Удерживая клавишу Ctrl, нажмите клавишу C.

Глава 1. Что делает шлюз

У вас есть устройство (счётчик электроэнергии, терморегулятор, ПЛК …), которое говорит на MODBUS RTU по последовательной линии. И есть программа (SCADA, скрипт, `mbpoll` …), которая говорит на MODBUS TCP по сети. Шлюз стоит между ними и переводит, как показано на рисунке 1.

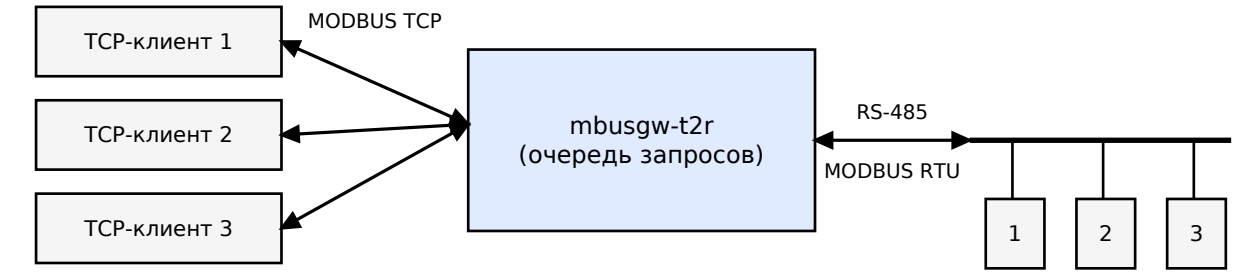


Рисунок 1. Путь данных через шлюз

Несколько TCP-клиентов могут подключаться одновременно: шлюз выстраивает их в очередь, чтобы на линию уходил только один запрос за раз — линия RTU физически не умеет нести две транзакции сразу.

На одной линии RS-485 может сидеть несколько устройств. Они различаются адресом ведомого внутри самого MODBUS-запроса; шлюзу их адреса знать не нужно вовсе.

Глава 2. Перед началом

Пройдите этот список один раз — он сэкономит час гаданий потом.

1. Шлюз установлен (см. README.md). После установки у вас есть программа `/opt/mbusgw-t2r/sbin/mbusgw-t2r` и файл настроек `/opt/mbusgw-t2r/etc/modbus-t2r_settings.conf`.

2. Вы знаете, к какому последовательному порту подключено устройство. В Linux это файл вида `/dev/ttyS0` или `/dev/ttyUSB0`. Если не уверены — выдерните и вставьте USB-адаптер и выполните:

```
$ dmesg | tail
```

3. Вы знаете параметры линии устройства: скорость, биты данных, чётность, стоп-биты. Они написаны в паспорте устройства; самый распространённый набор — `9600, 8, N, 1`.

4. Ваш пользователь имеет право открыть порт. Проверьте группу файла порта:

```
$ ls -l /dev/ttyUSB0
crw-rw---- 1 root dialout 188, 0 ... /dev/ttyUSB0
```

Если группа `dialout` — добавьте себя в неё и перезайдите в систему (либо просто запускайте шлюз через `sudo`):

```
$ sudo usermod -a -G dialout $USER
```

Глава 3. Файл настроек

В файле два раздела: `serials` (последовательные линии) и `listeners` (TCP-порты). Минимальный рабочий пример:

```
serials = (
  {  device = "/dev/ttyUSB0";
     chars  = "9600, 8, N, 1";
     iotmo  = 1000;
  }
);

listeners = (
  {  bind   = "TCP:0.0.0.0:502";
     target = "/dev/ttyUSB0";
     connlm = 8;
  }
);
```

Читается так: «открой порт /dev/ttyUSB0 на 9600-8-N-1 и жди ответа до 1000 мс; слушай TCP-клиентов на всех интерфейсах, порт 502, и пересылай их на этот порт».

3.1. Раздел serials

По одной записи на каждый последовательный порт. Таблица 3-1 перечисляет все ключи.

Ключ	Обяз.	Значение и допустимые величины
`device`	да	Файл последовательного порта, например /dev/ttyUSB0.
`chars`	да	Параметры линии: скорость, биты данных, чётность, стоп-биты. Скорость 50..4000000; биты данных 5..8; чётность N (нет), E (чётная), O (нечётная); стоп-биты 1..2.
`iotmo`	нет	Сколько ждать ответа устройства, миллисекунды. По умолчанию 1000.
`rs485`	нет	1 — просить ядро управлять направлением RS-485 (только для портов, которые это умеют). По умолчанию 0.
`ts_enabled`	нет	1 — включить псевдоустройство меток времени (глава 5). По умолчанию 0.
`ts_unit`	нет	Адрес ведомого, на который отвечает псевдоустройство. По умолчанию 135.
`ts_fncode`	нет	Код функции, на который оно отвечает. По умолчанию 4.
`ts_base_reg0`	нет	Первый регистр метки времени. По умолчанию 135.

Если запись неправильная, шлюз пропускает её и говорит почему — с допустимым диапазоном прямо в сообщении:

```
%T2R-E:  [serial #00:</dev/ttyUSB0>] --- speed 31 baud is out of range [50..4000000]
```

3.2. Раздел listeners

Ключ	Обяз.	Значение и допустимые величины
`bind`	да	Где слушать: TCP:<IP-адрес>:<порт>. Порт 1..65535; 0.0.0.0 — все интерфейсы. UDP не поддерживается.
`target`	да	На какой порт пересылать. Должен дословно, символ в символ, совпадать с device из serials.

Ключ	Обяз.	Значение и допустимые величины
`connlm`	нет	Сколько TCP-клиентов может ждать в очереди подключения, например 8.
`iotmo`	нет	Сетевой таймаут ввода-вывода, миллисекунды.

Порт 502 — стандартный порт MODBUS TCP, но в Linux порты ниже 1024 требуют прав root. Не хотите работать под root — возьмите, например, `5502` и укажите его в клиенте. Несколько listeners могут разделять один target: шлюз выстроит их запросы в очередь.

Глава 4. Запуск шлюза

Вручную, в терминале (сообщения идут прямо на экран):

```
$ /opt/mbusgw-t2r/sbin/mbusgw-t2r /settings=/opt/mbusgw-t2r/etc/modbus-t2r_settings.conf
```

Опция	Смысл
`/settings=<файл>`	Файл настроек (глава 3).
`/trace`	Подробная трассировка: каждый запрос и ответ печатается в hex. Бесценно при первом запуске.
`/logfile=<файл>`	Писать журнал в файл вместо экрана.
`/logsize=<октет>`	Ротировать файл журнала при превышении размера.

Здоровый запуск печатает (сокращённо):

```
%T2R-I-REVISNF, Rev: T2R X.00-06/aarch64(built at ...) (REV: 00.06.00)
%T2R-I:  Added device #00 [</dev/ttyUSB0>, Chars: <9600, 8, N, 1>, ...] --- added
%T2R-I:  Added listener #00 [Target: </dev/ttyUSB0>, Net: <TCP:0.0.0.0:502>, ...] --- added
%T2R-S-DEVREADY, Device </dev/ttyUSB0> [9600 baud, answer tmo: 1000 msec, t3.5: 4010 usec] --- is r
%T2R-S-LSNRRDY, [#3] Listener 0.0.0.0:502 [Target: </dev/ttyUSB0>] --- is ready
```

Важнее всего две строки: DEVREADY (порт открыт) и LSNRRDY (TCP-порт слушает). Видите обе — шлюз работает. Рисунок 2 показывает, что происходит дальше с каждым запросом.

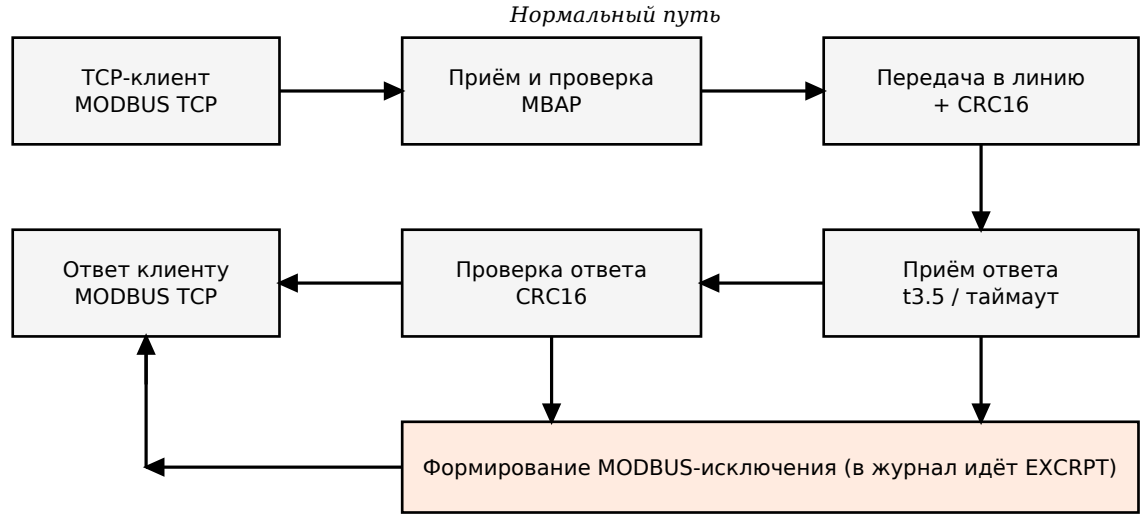


Рисунок 2. Жизненный цикл одного запроса

Остановка: один раз Ctrl/C и секунда ожидания. Переключить трассировку у уже работающего шлюза без перезапуска:

```
$ kill -USR1 <pid>
```

4.1. Первая проверка с mbpoll

`mbpoll` — маленький свободный MODBUS-клиент (`sudo apt install mbpoll`). Читаем 3 регистра хранения с устройства с адресом 1:

```
$ mbpoll -a 1 -r 1 -c 3 -1 127.0.0.1 -p 502
```

Если вернулись числа — всё работает, из конца в конец.

Глава 5. Псевдоустройство меток времени

Шлюз умеет отвечать сам — не трогая линию — на запрос чтения, посланный на специальный адрес ведомого. Ответ — текущее время машины, на которой работает шлюз. Так SCADA может читать точное время по обычному MODBUS.

При `ts_enabled = 1; ts_unit = 135; ts_fncode = 4; ts_base_reg0 = 135;` запрос «прочитай N входных регистров начиная со 135-го у ведомого 135» возвращает регистры рисунка 3.

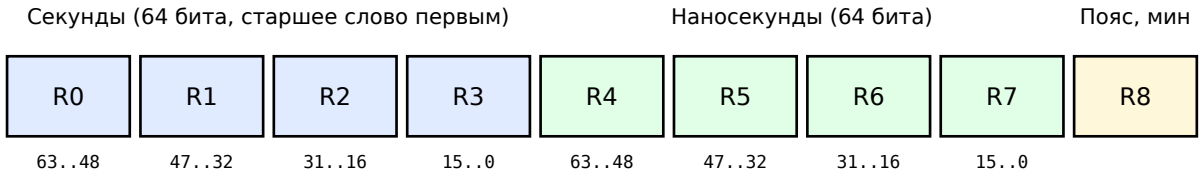


Рисунок 3. Раскладка регистров метки времени

Запрошено регистров	Что получаете
4	Секунды эпохи UNIX, 64 бита, старшее 16-битное слово первым.
8	Плюс наносекунды, упакованные так же в R4..R7.
9	Плюс R8 — смещение местного часового пояса, в знаковых минутах (Москва = +180).

Любое другое число регистров в TS-запросе получает MODBUS-исключение ILLEGAL_DATA_ADDRESS — это сделано намеренно.

Глава 6. Как читать журнал

Каждое важное событие — одна строка с кодом:

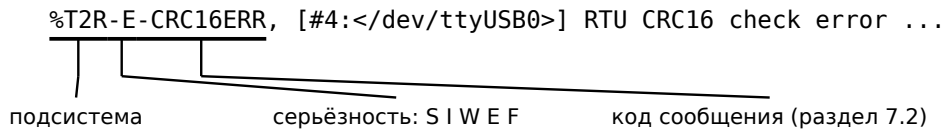


Рисунок 4. Анатомия сообщения журнала

Буква серьёзности: S — успех, I — информация, W — предупреждение, E — ошибка, F — фатально. Часть `[#4:</dev/ttyUSB0>]` означает: файловый дескриптор #4, вот этот последовательный порт.

Ищите в журнале по коду — именно для этого коды и существуют:

```
$ grep CRC16ERR /var/log/mbusgw-t2r.log
```

Глава 7. Поиск неисправностей

Золотое правило: запустите с /trace и читайте журнал. Шлюз всегда говорит, что именно ему не нравится.

7.1. Симптом, причина, действие

Вы видите	Что это значит / что делать
`%T2R-E-DEVOPNERR, ... errno: 2`	Файла порта нет: адаптер выдернут или имя неверное. Выполните dmesg | tail после втыкания; поправьте device в настройках.
`%T2R-E-DEVOPNERR, ... errno: 13`	Нет прав доступа. Запустите под sudo или добавьте себя в группу dialout.
`%T2R-E-DEVOPNERR, ... errno: 16`	Порт занят. Найдите держателя: sudo fuser /dev/ttyUSB0, и остановите его.
`%T2R-E-NOANSWER, ...`	Запрос ушёл, ответа нет: 1) неверный адрес ведомого в запросе клиента; 2) провода (поменяйте A/B у RS-485); 3) не та скорость/чётность — перепроверьте chars; 4) устройство выключено.
`%T2R-E-CRC16ERR, ...`	Байты приходят повреждёнными: 1) chars не совпадает с устройством (самое частое!); 2) помехи — земля, терминаторы, длина кабеля; 3) два мастера на одной линии.
`%T2R-E-BADFRAME, ...`	Что-то пришло, но это не осмысленный MODBUS-кадр: обычно те же причины, что у CRC16ERR; 400+ октетов мусора — на линии чужое, не-MODBUS устройство.
`CRC16ERR / BADFRAME время от времени, только на USB-адаптере FTDI`	Адаптер отдаёт ответ кусками, раз в 16 мс (latency_timer). С X.00-10 шлюз дожидается всего ответа для функций 01-06, 15, 16, 23; остальные по-прежнему обрезаются. Поставьте latency_timer в 1 мс (ответы заодно станут быстрее): скопируйте /opt/mbusgw-t2r/share/doc/mbusgw-t2r/99-ftdi-latency.rules в /etc/udev/rules.d/, выполните sudo udevadm control --reload && sudo udevadm trigger --subsystem-match=usb-serial; cat /sys/bus/usb-serial/devices/ttyUSB0/latency_timer должна напечатать 1.
`%T2R-W-FRAMETMO, ...`	Ответ начался, но не завершился: очень медленное или зависшее устройство. Увеличьте iotmo; проверьте кабель.
`%T2R-W-EXCRPT, ... (SERVER_DEVICE_FAILURE)`	Шлюз сообщил клиенту «на последовательной стороне сбой». Настоящая причина — в предыдущей строке журнала (NOANSWER, CRC16ERR ...).
`%T2R-E-LSNRERR, ... errno: 98`	TCP-порт занят: второй экземпляр шлюза или другая программа. Найдите: sudo ss -tlnp | grep 502.
`%T2R-E-LSNRERR, ... errno: 13`	Порты ниже 1024 требуют root. Запустите под sudo или возьмите порт выше 1024 (например 5502).
`... out of range [a..b]`	Значение в настройках вне диапазона; допустимый диапазон написан прямо в сообщении. Исправьте значение.
`No serials has been defined!`	Ни одна запись serials не прошла проверку. Читайте строки ошибок выше — каждая отброшенная запись объясняет причину.

Вы видите	Что это значит / что делать
`Данные неверные или сдвинутые`	Путаница порядка байт у клиента: регистры MODBUS — 16-битные слова big-endian; проверьте, как клиент собирает 32/64-битные значения.

7.2. Справочник кодов сообщений

Код	Сер.	Когда появляется
`REVISNF`	I	При старте: версия программы. Указывайте её, когда просите помощи.
`DEVREADY`	S	Последовательный порт открыт; показаны фактическая скорость, таймаут ответа и интервал t3.5.
`LSNRRDY`	S	TCP-порт слушает; показаны адрес, порт и целевое устройство.
`NETCONN`	S	Подключился TCP-клиент; показаны его адрес:порт и listener.
`NETDISCN`	S	TCP-клиент отключился; показаны его адрес:порт.
`DEVOPNERR`	E	Последовательный порт не открывается; errno объясняет почему (2 — файла нет, 13 — права, 16 — занят).
`LSNRERR`	E	Отказ bind()/listen() для TCP-порта; errno объясняет почему (98 — занят, 13 — привилегии).
`NOANSWER`	E	Устройство не ответило за таймаут iotmo.
`FRAMETMO`	W	Ответ начался, но не был завершён за таймаут.
`BADFRAME`	E	Длина принятого кадра вне разумного диапазона [5..256] октетов.
`CRC16ERR`	E	Контрольная сумма принятого кадра не сходится; показаны оба значения.
`EXCRPT`	W	TCP-клиенту возвращается MODBUS-исключение; показаны код и его имя.
`EXITST`	I	Шлюз завершается; показаны флаг выхода и итоговый статус.

7.3. Если ничего не помогло

Соберите и приложите к вопросу: 1) полный журнал запуска с /trace (от REVISNF до первой ошибки); 2) ваш файл настроек; 3) вывод `ls -l <устройство>` и `dmesg | tail -20`; 4) точную модель RTU-устройства и его паспортные параметры линии. С этими четырьмя вещами проблема почти всегда видна с первого взгляда.